mini-DM — instrukcja wdrożenia na serwer demo

Wersja aplikacji	commit `95062ff` (gałąź `main`)
Poprzednio wdrożona wersja	sprzed migracji nr 4 (wdrożenie z 2026-10-25)
Data przygotowania	2026-11-08
Środowisko	serwer demo klienta (`<ADRES_DEMO>`), Docker + docker compose, reverse proxy nginx

1. Co zawiera to wdrożenie

1. **Nowe pole „Badania laboratoryjne od dostawcy"** (`supplier_lab_results`) w formularzu edycji zamówienia oraz przeniesienie pola „Zablokowane" na dół formularza.
2. **Migracja bazy nr 4**, która **przebudowuje tabelę** `purchase_orders` : tworzy nową tabelę, kopiuje dane, usuwa starą i odtwarza indeksy. Istniejące zamówienia dostają `supplier_lab_results = 0` . Migracja uruchamia się automatycznie przy pierwszym starcie nowej wersji, w jednej transakcji. Jeśli się nie powiedzie, zmiany są wycofywane i aplikacja nie wystartuje.
3. **Poszerzony layout** stron po zalogowaniu. Daty ETA na liście zamówień już się nie zawijają, a na liście jest nowa kolumna z ikoną kolby (badania od dostawcy).

Nie zmieniają się zmienne środowiskowe, port (4321) ani konfiguracja `docker-compose.yml` i nginx.

> ***Ważne:*** *to pierwsza migracja, która przebudowuje tabelę zamiast tylko dodać kolumnę.* ***Kopia zapasowa bazy (krok 4) jest obowiązkowa.***

2. Oznaczenia używane w instrukcji

Symbol	Znaczenie
`<KATALOG_ZRODEL>`	katalog na serwerze, w którym budowany jest obraz (kopia repozytorium)
`<KATALOG_COMPOSE>`	katalog z plikiem `docker-compose.yml`, w którym zdefiniowana jest usługa `mini-dm`
`<KATALOG_DANYCH>`	katalog z bazą SQLite, podmontowany w kontenerze jako `/app/data`
`<KATALOG_KOPII>`	dowolny katalog na kopie zapasowe, poza `<KATALOG_DANYCH>`

Instrukcja zakłada, że usługa `mini-dm` w `docker-compose.yml` korzysta z obrazu `mini-dm:latest` , tak jak przy poprzednim wdrożeniu.

3. Przygotowanie paczki ze źródłami (lokalnie)

Na komputerze deweloperskim, w katalogu projektu:

```
git archive --format=zip -o mini-dm-95062ff.zip 95062ff
```

`git archive` pakuje tylko pliki śledzone przez git. W paczce nie ma więc `node_modules` , `dist` , `.astro` , `data` , `.env*` ani lokalnych konfiguracji.

Skopiuj `mini-dm-95062ff.zip` na serwer. Następnie wyczyść `<KATALOG_ZRODEL>` ze starych plików źródłowych i rozpakuj tam paczkę:

```
cd <KATALOG_ZRODEL>
unzip -o ~/mini-dm-95062ff.zip
```

> *Nigdy nie rozpakowuj paczki do `<KATALOG_DANYCH>` i nie czyść tego katalogu.*

4. Zatrzymanie aplikacji i kopia zapasowa bazy

4.1. Zachowaj obecny obraz pod osobną nazwą, żeby można było do niego wrócić:

```
docker tag mini-dm:latest mini-dm:prev
```

4.2. Zatrzymaj aplikację. SQLite pozwala na jednego zapisującego, a kopia musi być spójna:

```
cd <KATALOG_COMPOSE>
docker compose stop mini-dm
```

4.3. Zrób kopię **wszystkich** plików bazy, czyli `mini-dm.db` oraz, jeśli istnieją, `mini-dm.db-wal` i `mini-dm.db-shm` :

```
STAMP=$(date +%Y%m%d-%H%M)
mkdir -p <KATALOG_KOPII>/mini-dm-$STAMP
cp -a <KATALOG_DANYCH>/mini-dm.db* <KATALOG_KOPII>/mini-dm-$STAMP/
ls -l <KATALOG_KOPII>/mini-dm-$STAMP/
```

4.4. Zanotuj stan bazy przed migracją. Polecenie uruchamia na chwilę stary obraz w trybie tylko do odczytu:

```
docker run --rm -v <KATALOG_DANYCH>:/app/data mini-dm:prev node -e '
const db = require("better-sqlite3")("/app/data/mini-dm.db", { readonly: true,
fileMustExist: true });
console.log("zamowienia:", db.prepare("SELECT COUNT(*) AS c FROM purchase_orders").get().c);
console.log("historia:  ", db.prepare("SELECT COUNT(*) AS c FROM
purchase_order_history").get().c);
console.log("migracje:  ", db.prepare("SELECT version FROM schema_migrations ORDER BY
version").all().map(r => r.version).join(","));
'
```

Zapisz liczby `zamowienia` i `historia`. Pole `migracje` powinno pokazać `1,2,3`.

5. Budowa nowego obrazu

```
cd <KATALOG_ZRODEL>
docker build -t mini-dm:latest .
```

Budowa powinna zakończyć się bez błędów. Etap `npm run build` jest w niej wymagany.

6. Kontrola uprawnień do plików bazy

Kontener działa jako nieuprzywilejowany użytkownik `app`. Przy poprzednim wdrożeniu pliki bazy należały do innego użytkownika z prawami `644`. Aplikacja wtedy startowała i wyświetlała dane, ale **pierwszy zapis kończył się błędem**.

```
ls -l <KATALOG_DANYCH>/
```

Jeśli pliki `mini-dm.db*` nie mają praw zapisu dla „others" (`-rw-rw-rw-`), wykonaj:

```
chmod 666 <KATALOG_DANYCH>/mini-dm.db*
```

Sam katalog również musi być zapisywalny dla kontenera (wcześniej miał prawa `777`).

7. Uruchomienie nowej wersji

```
cd <KATALOG_COMPOSE>
docker compose up -d mini-dm
docker compose logs --tail 50 mini-dm
```

W logach **nie** powinno być błędów typu `Migration 4 ...` , `SQLITE_...` ani `foreign key violation` .
Migracja nie wypisuje komunikatu o sukcesie, więc brak błędów i działający serwer oznaczają, że się udała.

8. Weryfikacja

8.1. Baza. Uruchom to samo sprawdzenie co w kroku 4.4, tym razem na nowym obrazie:

```
docker run --rm -v <KATALOG_DANYCH>:/app/data mini-dm:latest node -e '
const db = require("better-sqlite3")("/app/data/mini-dm.db", { readonly: true,
fileMustExist: true });
console.log("zamowienia:", db.prepare("SELECT COUNT(*) AS c FROM purchase_orders").get().c);
console.log("historia:  ", db.prepare("SELECT COUNT(*) AS c FROM
purchase_order_history").get().c);
console.log("migracje:  ", db.prepare("SELECT version FROM schema_migrations ORDER BY
version").all().map(r => r.version).join(","));
console.log("kolumny:   ", db.prepare("PRAGMA table_info(purchase_orders)").all().map(c =>
c.name).join(", "));
'
```

Oczekiwany wynik:

- `zamowienia` i `historia` mają **te same liczby** co w kroku 4.4,
- `migracje` pokazuje `1,2,3,4` ,
- w `kolumny` pole `supplier_lab_results` jest zaraz po `test_results` , a `is_blocked` zaraz po `notes` .

8.2. Aplikacja w przeglądarce (`https://<ADRES_DEMO>`):

1. Zaloguj się. Strona Dashboard powinna się wyświetlić, a strony powinny być nieco szersze niż wcześniej.

2. Otwórz **Zamówienia**. Daty „ETA port" i „ETA cel" są w jednej linii, a po „ETA cel" jest kolumna z ikoną kolby (na razie pusta).
3. **Test zapisu (obowiązkowy):** otwórz edycję dowolnego zamówienia. Pole „Zablokowane" jest na dole formularza, pod „Uwagami". Zaznacz „Badania laboratoryjne od dostawcy" i zapisz.
4. Wróć do listy. Przy tym zamówieniu powinna być zielona kolba.
5. Ponownie otwórz edycję tego zamówienia. W historii zmian powinien być wpis „Badania laboratoryjne od dostawcy: Nie → Tak".
6. Jeśli to zamówienie testowe klienta, odznacz pole i zapisz ponownie, żeby przywrócić poprzedni stan.

Wdrożenie jest zakończone dopiero po udanym teście zapisu (punkty 3–5), a nie po samym wyświetleniu strony.

9. Wycofanie zmian (rollback)

Wykonaj te kroki tylko wtedy, gdy krok 7 lub 8 się nie powiedzie:

```
cd <KATALOG_COMPOSE>
docker compose stop mini-dm

# przywrócenie bazy z kopii (usuń bieżące pliki -wal/-shm, żeby nie zostały po nowej wersji)
rm -f <KATALOG_DANYCH>/mini-dm.db-wal <KATALOG_DANYCH>/mini-dm.db-shm
cp -a <KATALOG_KOPII>/mini-dm-<STAMP>/mini-dm.db* <KATALOG_DANYCH>/
chmod 666 <KATALOG_DANYCH>/mini-dm.db*

# powrót do poprzedniego obrazu
docker tag mini-dm:prev mini-dm:latest
docker compose up -d mini-dm
docker compose logs --tail 50 mini-dm
```

Po rollbacku powtórz sprawdzenie z kroku 4.4 na obrazie `mini-dm:latest`. Powinno ono pokazać `migracje: 1,2,3` i te same liczby co przed wdrożeniem.

> *Przywróć **bazę i obraz razem**. Po rollbacku dane wprowadzone w nowej wersji, między krokiem 7 a rollbackiem, zostaną utracone.*

10. Po udanym wdrożeniu

- Zostaw kopię z kroku 4.3 i obraz `mini-dm:prev` przez kilka dni. Gdy klient potwierdzi, że wszystko działa, możesz usunąć obraz poleceniem `docker image rm mini-dm:prev`.
- Usuń paczkę `mini-dm-95062ff.zip` z serwera.